Университет ИТМО

Факультет программной инженерии и компьютерной техники

Лабораторная работа № 2

«Последовательный интерфейс UART»
Дисциплина: Проектирование вычислительных систем

Вариант: **1. «Светофор»**

Выполнили студенты группы P3410:
Нестеров Владислав Алексеевич
Джохадзе Анна Бекаевна

Преподаватель:
Пинкевич Василий Юрьевич

Санкт-Петербург, 2026

1. Цель и задание работы

Цель работы — изучить передачу данных по UART и разработать драйвер для двух режимов: опроса и прерываний.

Для варианта 1 требуется добавить к светофору управление через UART: включение и отключение кнопки, изменение длительности красного, запрос состояния и переключение режима обмена. Скорость — **57600 бит/с**. Каждый принятый символ возвращается в терминал как эхо. Ответы выводятся с новой строки; команды настройки подтверждаются `ok`, неизвестные команды — сообщением об ошибке.

Приём в обоих режимах и передача в режиме прерываний должны работать без ожидания данных или окончания передачи. В режиме опроса прерывания UART должны быть выключены. Программа работает без операционной системы реального времени.

2. Подключение и настройка UART

Используется стенд SDK-1.1M с STM32F427VIT6. USART6 соединён со встроенным USB–UART отладчика. На компьютере выбирается COM-порт интерфейса 1. Назначение выводов приведено в таблице 1.

Таблица 1 — Подключение элементов стенда

Сигнал	**Вывод**	**Настройка или уровень включения**
USART6_TX	PC6	Альтернативная функция AF8, выход push-pull
USART6_RX	PC7	AF8, подтяжка вверх
Кнопка	PC15	Вход; нажатие соответствует 0
Зелёный	PD13	Включается при 1
Жёлтый	PD14, PD15	PD14 = 1, PD15 = 0
Красный	PD14, PD15	PD14 = 0, PD15 = 1

Параметры UART: **57600 бит/с, 8 бит данных, без чётности, 1 стоп-бит (8N1), без управления потоком**. В терминале отключается Local echo, поскольку символы возвращает сама плата.

Источник тактирования — HSI 16 МГц, PLL отключён. Делители AHB, APB1 и APB2 равны 1, поэтому частота USART6 составляет 16 МГц. При передискретизации ×16 драйвер рассчитывает делитель:

$$BRR = \mathrm{round}(16\,000\,000 / 57\,600) = 278.$$

Расчётная скорость — около 57554 бит/с, отклонение — −0,08 %. Время светофора отсчитывается по миллисекундному SysTick.

3. Логика светофора

После сброса установлены `mode 1`, `timeout 4` и режим UART `I`. Светофор повторяет цикл, показанный на рисунке 1, где T — обычная длительность красного. Зелёный мигает четыре раза с полупериодом 250 мс; вся фаза мигания длится 2 с.

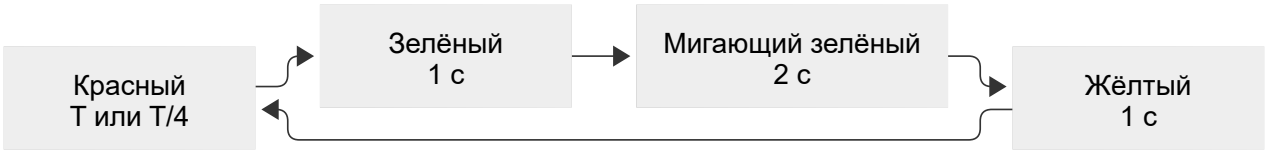


Рисунок 1 — Последовательность состояний светофора

В `mode 1` кнопка сокращает ближайший красный до четверти обычной длительности. Нажатие во время мигания или жёлтого запоминается до начала красного. При нажатии во время красного четверть периода отсчитывается от его начала; если она уже прошла, включается зелёный. Нажатие во время постоянного зелёного игнорируется. Порядок цветов сохраняется, а следующий красный без нового запроса снова длится полный период.

В `mode 2` новые нажатия игнорируются. Ранее принятый запрос сохраняется. Кнопка опрашивается в обоих режимах, дребезг подавляется за 30 мс. Удержание создаёт только одно событие нажатия.

Команда `set timeout X` задаёт красный период в целых секундах от 1 до 2147483. Новый срок применяется сразу и отсчитывается от начала текущего красного. Остальные фазы не меняются. Например, при X = 8 обычный красный длится 8 с, сокращённый — 2 с. После сброса настройки возвращаются к начальным.

4. Архитектура и главный цикл

Программа разделена на модули. Их связи показаны на рисунке 2, назначение — в таблице 2. Работа с регистрами и номерами выводов вынесена в драйверы.

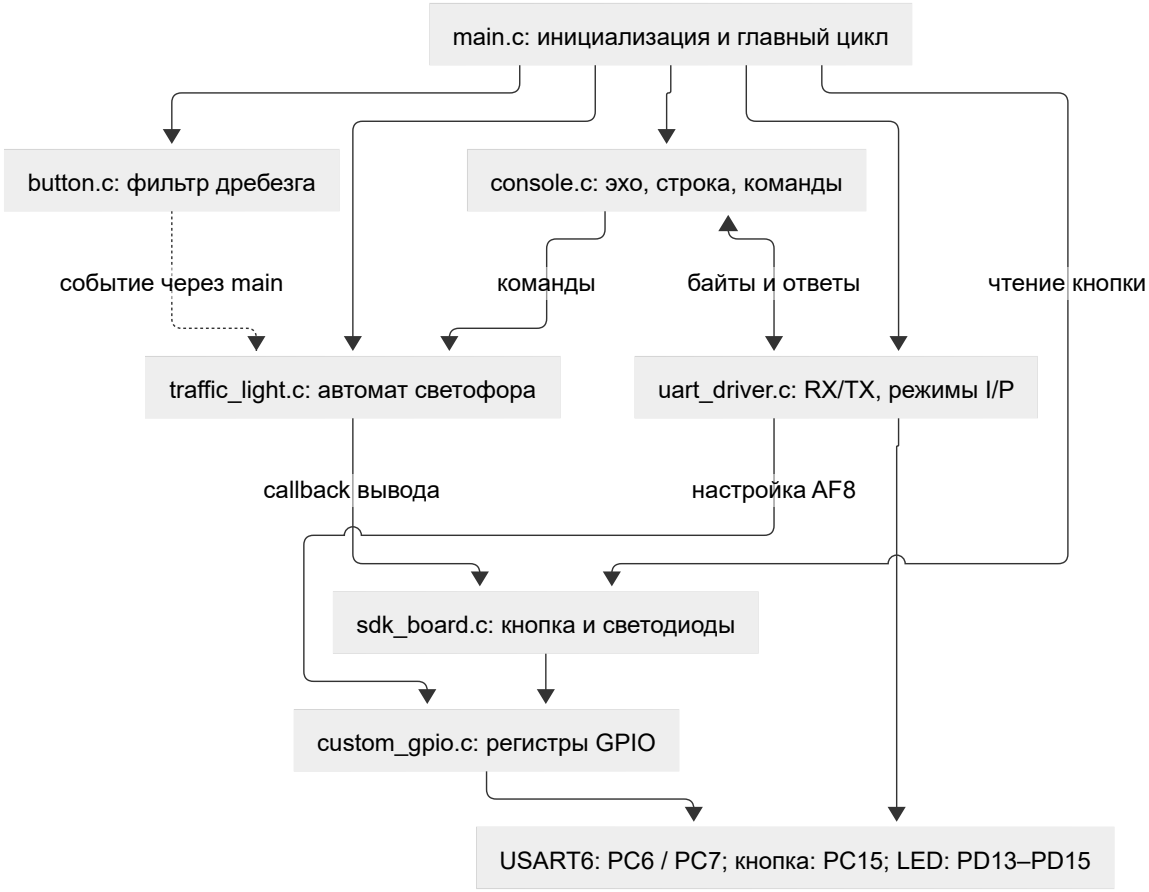


Рисунок 2 — Связи программных модулей

Таблица 2 — Назначение модулей

Модуль	**Назначение**
`main`	Запуск и главный цикл
`uart_driver`	Обмен через USART6, буферы, опрос и прерывания
`console`	Эхо, приём строки и выполнение команд
`traffic_light`	Состояния светофора, кнопка и длительности фаз
`button`	Подавление дребезга и события кнопки
`sdk_board`	Чтение кнопки и включение светодиодов стенда
`custom_gpio`	Настройка и управление выводами GPIO

Главный цикл показан на рисунке 3. Он обслуживает UART, получает время, обновляет светофор и кнопку, затем обрабатывает ввод. Консоль разбирает не более одного байта за итерацию. Ожидания ввода и `HAL_Delay()` нет, поэтому светофор продолжает работать во время набора команды.

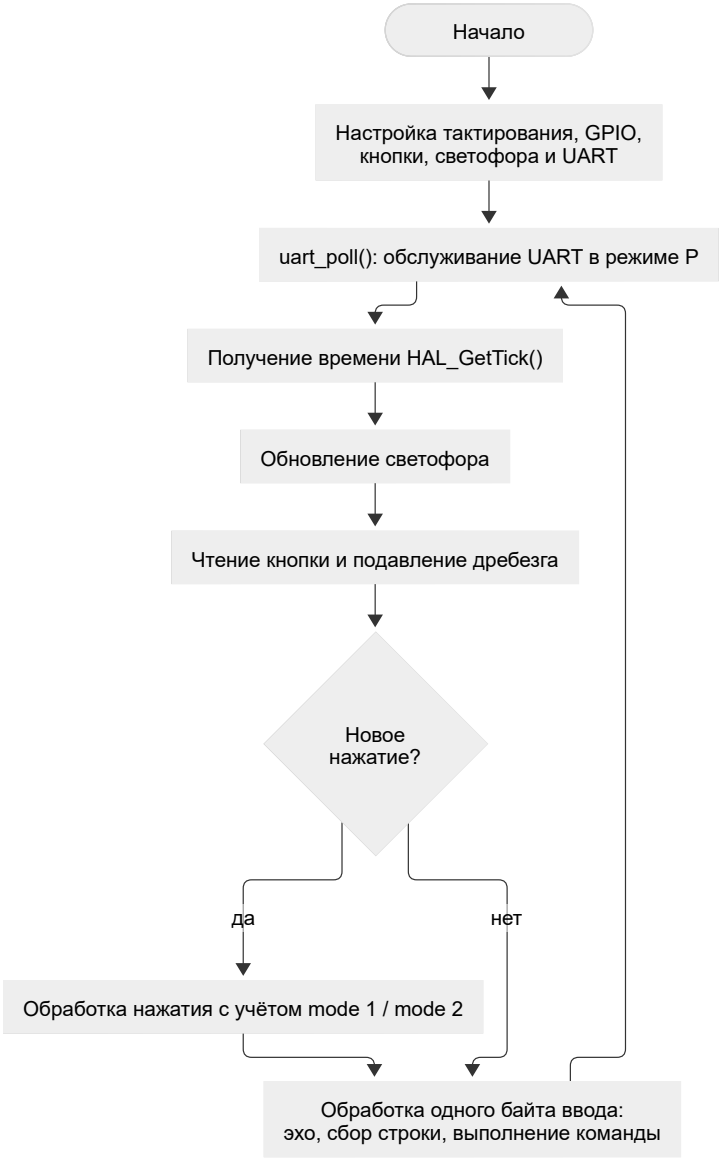


Рисунок 3 — Блок-схема главного цикла

5. Устройство драйвера UART

UART реализован собственным регистровым драйвером. HAL используется для запуска, настройки тактирования и получения времени; функции обмена HAL UART и DMA не применяются. Два режима используют общую функцию `service_hardware()`, которая проверяет готовность приёма и передачи.

В **режиме опроса P** эта функция вызывается из `uart_poll()` в главном цикле. Источники прерываний USART6 и его линия в NVIC выключены. Системный SysTick остаётся включённым для отсчёта времени.

В **режиме прерываний I** обслуживание выполняется в `USART6_IRQHandler()`. Флаг `RXNE` означает, что принят байт: он читается из `DR` и помещается в очередь RX. Флаг `TXE` означает готовность регистра `DR` принять следующий байт из очереди TX. Когда TX пуст, прерывание передачи выключается. Команды выполняются в главном цикле.

Кольцевые буферы RX и TX имеют размеры 512 и 2048 байт; полезная ёмкость — 511 и 2047 байт. Буферы позволяют принимать и передавать данные

независимо от выполнения команд. `uart_read_byte()` возвращает байт либо `false`, если данных нет. `uart_write()` помещает сообщение в очередь целиком либо возвращает `false`, если места недостаточно. Ожидания передачи нет.

При смене режима драйвер временно запрещает прерывания USART6, меняет способ обслуживания и включает нужные источники. Содержимое очередей сохраняется. Совместный доступ главного цикла и обработчика защищён короткими критическими секциями.

При ошибке приёма или переполнении RX устанавливается признак ошибки. Консоль отбрасывает текущую строку до Enter и сообщает `ERROR`. Если места для ответа недостаточно, разбор ввода откладывается. Для обычного обмена следующую команду отправляют после ответа на предыдущую.

6. Команды управления

Команды вводятся строчными латинскими буквами и завершаются Enter. Поддерживаются CR, LF и CRLF; пара CRLF завершает команду один раз. Каждый принятый символ возвращается в терминал. Команды и ответы приведены в таблице 3.

Таблица 3 — Команды варианта 1

Команда	**Действие и ответ**
`?`	Текущая фаза: `red`, `green`, `blinking green` или `yellow`; режим кнопки; тайм-аут; I или P. Пример: `red; mode 1; timeout 4; I`
`set mode 1`	Включить обработку кнопки; `OK`
`set mode 2`	Игнорировать новые нажатия; `OK`
`set timeout X`	Установить красный период X секунд; `OK`
`set interrupts on`	Включить режим прерываний I; `OK`
`set interrupts off`	Включить режим опроса P; `OK`
Неизвестная команда	`ERROR: unknown command`
Неверный тайм-аут	`ERROR: timeout must be an integer from 1 to 2147483 seconds`

Поле `timeout` содержит обычную длительность красного, а не оставшееся время. Отрицательные, дробные и слишком большие значения отклоняются без изменения настройки. Строка ограничена 95 символами; длинный или повреждённый ввод вызывает ошибку. Исправление ввода поддерживается клавишами Backspace и DEL.

7. Проверка программы

Программа собрана в STM32CubeIDE и загружена через DBG USB. Проверка проведена на стенде SDK-1.1M при непрерывном выполнении без остановок на точках останова. Результаты приведены в таблице 4. Все перечисленные сценарии выполнены успешно, отклонений не обнаружено.

Таблица 4 — Результаты проверки на стенде

Проверка	**Результат**
Сброс и команда `?`	Текущий цвет, mode 1, timeout 4, I; эхо введённых символов
Цикл и кнопка в mode 1	Красный 4 с, зелёный 1 с, мигание 2 с, жёлтый 1 с; по кнопке ближайший красный сокращается до 1 с
`set mode 2`, затем `set mode 1`	Ответы OK; новые нажатия сначала игнорируются, затем снова обрабатываются
`set timeout 8`	Ответ OK, статус timeout 8; обычный красный 8 с, сокращённый — 2 с
Переключение off → on	Ответы OK; статус меняется P → I; команды, эхо и светофор работают в обоих режимах
Неизвестная команда; неверный X; длинная строка	Сообщение ERROR; настройки сохраняются; следующая правильная команда принимается
Завершение CR, LF и CRLF	Один ответ на каждую завершённую команду

8. Выводы

В светофор добавлено управление через UART на скорости 57600 бит/с. Реализованы все команды варианта 1, эхо и сообщения об ошибках. Драйвер поддерживает опрос и прерывания, буферизацию и обмен без блокирующих ожиданий. Переключение режимов сохраняет очереди данных, а приём команд не останавливает работу светофора.

Проверка на стенде подтвердила работу команд, эха, кнопки и светофора в обоих режимах UART. В ходе работы изучены обслуживание UART по флагам и прерываниям, работа кольцевых буферов и защита данных при совместном доступе из главного цикла и обработчика.

9. Исходные коды собственной разработки

Ниже приведены прикладная часть `main.c`, драйвер UART, консоль и доработанный автомат светофора. Код HAL, CMSIS, сгенерированная настройка тактирования и неизменённые модули первой лабораторной не включены.

9.1. Прикладная часть основной программы: main.c

```
#include "main.h"

#include "button.h"
#include "console.h"
#include "sdk_board.h"
#include "traffic_light.h"
#include "uart_driver.h"

static Button button;
static TrafficLight traffic_light;
static Console console;

void SystemClock_Config(void);

static void set_traffic_lights(bool red, bool yellow, bool green)
{
    uint8_t mask = SDK_LED_NONE;
    mask |= red ? SDK_LED_RED : 0U;
    mask |= yellow ? SDK_LED_YELLOW : 0U;
    mask |= green ? SDK_LED_GREEN : 0U;
    sdk_board_write_leds(mask);
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    sdk_board_init();
    const uint32_t now_ms = HAL_GetTick();
    button_init(&button, sdk_board_button_pressed(), now_ms, 30U);
    traffic_light_init(&traffic_light, now_ms, set_traffic_lights);
    if (!uart_init()) {
        Error_Handler();
    }
    console_init(&console);

    while (1) {
        uart_poll();
        const uint32_t current_ms = HAL_GetTick();
        traffic_light_update(&traffic_light, current_ms);
        const ButtonEvent event = button_update(&button, sdk_board_button_pressed(), current_ms);
        if (event.type == BUTTON_EVENT_PRESSED) {
            traffic_light_on_button_pressed(&traffic_light, current_ms);
        }
        console_update(&console, &traffic_light, current_ms);
    }
}
```

9.2. Интерфейс UART: uart_driver.h

```
#ifndef UART_DRIVER_H
#define UART_DRIVER_H

#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

enum {
    UART_BAUD_RATE = 57600U,
    UART_RX_BUFFER_SIZE = 512U,
    UART_TX_BUFFER_SIZE = 2048U
};

bool uart_init(void);
/* Polling mode: one bounded RX/TX service pass. Interrupt mode: no-op. */
void uart_poll(void);
bool uart_read_byte(uint8_t *byte);
/* Enqueue the entire message or return false; never wait for transmission. */
bool uart_write(const char *data, size_t length);
size_t uart_tx_free(void);
void uart_set_interrupts(bool enabled);
bool uart_interrupts_enabled(void);
/* Report and clear a hardware RX error or a software RX buffer overflow. */
bool uart_take_rx_error(void);
void USART6_IRQHandler(void);

#endif
```

9.3. Драйвер USART6: опрос и прерывания: uart_driver.c

```
#include "uart_driver.h"

#include "custom_gpio.h"
#include "stm32f4xx_hal.h"

#define RX_MASK (UART_RX_BUFFER_SIZE - 1U)
#define TX_MASK (UART_TX_BUFFER_SIZE - 1U)
#define RX_ERRORS (USART_SR_ORE | USART_SR_NE | USART_SR_FE | USART_SR_PE)

_Static_assert((UART_RX_BUFFER_SIZE & RX_MASK) == 0U, "RX size must be a power of two");
_Static_assert((UART_TX_BUFFER_SIZE & TX_MASK) == 0U, "TX size must be a power of two");

static uint8_t rx_data[UART_RX_BUFFER_SIZE];
static uint8_t tx_data[UART_TX_BUFFER_SIZE];
static volatile uint32_t rx_head;
static volatile uint32_t rx_tail;
static volatile uint32_t tx_head;
static volatile uint32_t tx_tail;
static volatile bool rx_error;
static volatile bool use_interrupts;

static uint32_t lock(void)
{
    const uint32_t previous = __get_PRIMASK();
    __disable_irq();
    return previous;
}

static void unlock(uint32_t previous)
{
    __set_PRIMASK(previous);
}

/* Called only from the USART ISR or a short critical section in main. */
static void service_hardware(void)
{
    const uint32_t status = USART6->SR;
    if ((status & (USART_SR_RXNE | RX_ERRORS)) != 0U) {
        /* RM0090: SR read followed by DR read clears RXNE and RX errors. */
        const uint8_t byte = (uint8_t)USART6->DR;
        if ((status & RX_ERRORS) != 0U) {
            rx_error = true;
        }
        if ((status & USART_SR_RXNE) != 0U
            && (status & (USART_SR_NE | USART_SR_FE | USART_SR_PE)) == 0U) {
            const uint32_t next = (rx_head + 1U) & RX_MASK;
            if (next != rx_tail) {
                rx_data[rx_head] = byte;
                __DMB();
                rx_head = next;
            } else {
                rx_error = true;
            }
        }
    }

    if ((status & USART_SR_TXE) != 0U) {
        if (tx_tail != tx_head) {
            USART6->DR = tx_data[tx_tail];
            tx_tail = (tx_tail + 1U) & TX_MASK;
        }
        if (tx_tail == tx_head) {
            USART6->CR1 &= ~USART_CR1_TXEIE;
        }
    }
}

bool uart_init(void)
{
    NVIC_DisableIRQ(USART6_IRQn);
    RCC->APB2ENR |= RCC_APB2ENR_USART6EN;
    (void)RCC->APB2ENR;
    RCC->APB2RSTR |= RCC_APB2RSTR_USART6RST;
    RCC->APB2RSTR &= ~RCC_APB2RSTR_USART6RST;

    rx_head = rx_tail = tx_head = tx_tail = 0U;
    rx_error = false;
    use_interrupts = false;

    /* USART6 is on APB2, oversampling by 16, 8 data bits, no parity, 1 stop bit. */
    USART6->CR1 = 0U;
    USART6->CR2 = 0U;
    USART6->CR3 = 0U;
    USART6->BRR = (HAL_RCC_GetPCLK2Freq() + UART_BAUD_RATE / 2U) / UART_BAUD_RATE;
    USART6->CR1 = USART_CR1_UE | USART_CR1_TE | USART_CR1_RE;

    const CustomGpioConfig tx_config = {
        .mode = CUSTOM_GPIO_MODE_ALTERNATE,
        .output_type = CUSTOM_GPIO_PUSH_PULL,
```

```
        .speed = CUSTOM_GPIO_SPEED_LOW,
        .pull = CUSTOM_GPIO_NO_PULL,
        .alternate_function = 8U,
        .output_high = true
    };
    CustomGpioConfig rx_config = tx_config;
    rx_config.pull = CUSTOM_GPIO_PULL_UP;
    if (!custom_gpio_init(GPIOC, (uint16_t)(1U << 6U), &tx_config)
        || !custom_gpio_init(GPIOC, (uint16_t)(1U << 7U), &rx_config)) {
        USART6->CR1 = 0U;
        return false;
    }

    NVIC_SetPriority(USART6_IRQn, 5U);
    NVIC_ClearPendingIRQ(USART6_IRQn);
    uart_set_interrupts(true);
    return true;
}

void uart_poll(void)
{
    if (!use_interrupts) {
        const uint32_t previous = lock();
        service_hardware();
        unlock(previous);
    }
}

bool uart_read_byte(uint8_t *byte)
{
    if (byte == NULL) {
        return false;
    }
    const uint32_t previous = lock();
    const bool available = rx_tail != rx_head;
    if (available) {
        *byte = rx_data[rx_tail];
        rx_tail = (rx_tail + 1U) & RX_MASK;
    }
    unlock(previous);
    return available;
}

size_t uart_tx_free(void)
{
    /* Only main produces TX data. ISR can only increase the free space. */
    return (size_t)((tx_tail - tx_head - 1U) & TX_MASK);
}

bool uart_write(const char *data, size_t length)
{
    if (data == NULL || length > uart_tx_free()) {
        return false;
    }
    uint32_t head = tx_head;
    for (size_t i = 0U; i < length; ++i) {
        tx_data[head] = (uint8_t)data[i];
        head = (head + 1U) & TX_MASK;
    }
    /* Publish only after copying. Keep interrupts enabled during the copy. */
    const uint32_t previous = lock();
    __DMB();
    tx_head = head;
    if (use_interrupts && tx_head != tx_tail) {
        USART6->CR1 |= USART_CR1_TXEIE;
    }
    unlock(previous);
    return true;
}

void uart_set_interrupts(bool enabled)
{
    const uint32_t previous = lock();
    NVIC_DisableIRQ(USART6_IRQn);
    USART6->CR1 &= ~(USART_CR1_RXNEIE | USART_CR1_TXEIE | USART_CR1_PEIE | USART_CR1_TCIE);
    USART6->CR3 &= ~USART_CR3_EIE;
    /* Preserve both rings and consume a possible pending RX byte before switching. */
    service_hardware();
    use_interrupts = enabled;
    NVIC_ClearPendingIRQ(USART6_IRQn);
    if (enabled) {
        USART6->CR3 |= USART_CR3_EIE;
        USART6->CR1 |= USART_CR1_RXNEIE;
        if (tx_tail != tx_head) {
            USART6->CR1 |= USART_CR1_TXEIE;
        }
        NVIC_EnableIRQ(USART6_IRQn);
    }
    unlock(previous);
}

bool uart_interrupts_enabled(void)
```

```
{
    return use_interrupts;
}

bool uart_take_rx_error(void)
{
    const uint32_t previous = lock();
    const bool error = rx_error;
    rx_error = false;
    unlock(previous);
    return error;
}

void USART6_IRQHandler(void)
{
    if (use_interrupts) {
        service_hardware();
    }
}
```

9.4. Структура консоли: console.h

```
#ifndef CONSOLE_H
#define CONSOLE_H

#include <stdbool.h>
#include <stddef.h>

#include "traffic_light.h"

typedef struct {
    char line[96];
    size_t length;
    bool discard_line;
    bool previous_cr;
    bool rx_failed;
} Console;

void console_init(Console *console);
void console_update(Console *console, TrafficLight *light, uint32_t now_ms);

#endif
```

9.5. Командный интерфейс: console.c

```
#include "console.h"

#include <stdio.h>
#include <string.h>

#include "uart_driver.h"

/* Enough space for echo, an error and the longest status response in one call. */
enum { CONSOLE_TX_RESERVE = 256U };

static void reply(const char *message)
{
    (void)uart_write("\r\n", 2U);
    (void)uart_write(message, strlen(message));
    (void)uart_write("\r\n", 2U);
}

static bool parse_seconds(const char *text, uint32_t *seconds)
{
    uint32_t value = 0U;
    if (*text == '\0') {
        return false;
    }
    while (*text != '\0') {
        if (*text < '0' || *text > '9') {
            return false;
        }
        const uint32_t digit = (uint32_t)(*text - '0');
        if (value > (TRAFFIC_MAX_TIMEOUT_SECONDS - digit) / 10U) {
            return false;
        }
        value = value * 10U + digit;
        ++text;
    }
    if (value == 0U) {
        return false;
    }
    *seconds = value;
    return true;
}
```

```
static void execute(char *line, TrafficLight *light, uint32_t now_ms)
{
    while (*line == ' ' || *line == '\t') {
        ++line;
    }
    size_t length = strlen(line);
    while (length > 0U && (line[length - 1U] == ' ' || line[length - 1U] == '\t')) {
        line[--length] = '\0';
    }
    if (length == 0U) {
        return;
    }

    if (strcmp(line, "?") == 0) {
        char status[128];
        (void)snprintf(status, sizeof status, "%s; mode %u; timeout %lu; %c",
            traffic_light_state_name(light), light->button_enabled ? 1U : 2U,
            (unsigned long)(light->red_ms / 1000U), uart_interrupts_enabled() ? 'I' : 'P');
        reply(status);
    } else if (strcmp(line, "set mode 1") == 0) {
        light->button_enabled = true;
        reply("OK");
    } else if (strcmp(line, "set mode 2") == 0) {
        light->button_enabled = false;
        reply("OK");
    } else if (strcmp(line, "set interrupts on") == 0) {
        uart_set_interrupts(true);
        reply("OK");
    } else if (strcmp(line, "set interrupts off") == 0) {
        uart_set_interrupts(false);
        reply("OK");
    } else if (strncmp(line, "set timeout ", 12U) == 0) {
        uint32_t seconds;
        if (parse_seconds(line + 12U, &seconds)
            && traffic_light_set_timeout(light, seconds, now_ms)) {
            reply("OK");
        } else {
            reply("ERROR: timeout must be an integer from 1 to 2147483 seconds");
        }
    } else {
        reply("ERROR: unknown command");
    }
}

void console_init(Console *console)
{
    console->length = 0U;
    console->discard_line = false;
    console->previous_cr = false;
    console->rx_failed = false;
    reply("SDK-1.1M lab 2: 57600 8N1; mode 1; timeout 4; I. Enter ? + Enter.");
}

void console_update(Console *console, TrafficLight *light, uint32_t now_ms)
{
    /* Leave RX queued until there is room for the echo and a full response. */
    if (uart_tx_free() < CONSOLE_TX_RESERVE) {
        return;
    }
    if (uart_take_rx_error()) {
        console->discard_line = true;
        console->rx_failed = true;
    }
    uint8_t byte;
    if (!uart_read_byte(&byte)) {
        return;
    }
    const char character = (char)byte;
    (void)uart_write(&character, 1U);

    const bool paired_lf = console->previous_cr && byte == '\n';
    console->previous_cr = byte == '\r';
    if (paired_lf) {
        return;
    }
    if (byte == '\r' || byte == '\n') {
        if (console->discard_line) {
            reply(console->rx_failed ? "ERROR: UART receive error; command discarded"
                                     : "ERROR: command too long or invalid character");
        } else {
            console->line[console->length] = '\0';
            execute(console->line, light, now_ms);
        }
        console->length = 0U;
        console->discard_line = false;
        console->rx_failed = false;
    } else if (byte == '\b' || byte == 127U) {
        if (!console->discard_line && console->length > 0U) {
            --console->length;
            (void)uart_write(" \b", 2U);
        }
    } else if (!console->discard_line) {
```

```
        if ((byte < 32U && byte != '\t') || byte > 126U
            || console->length >= sizeof console->line - 1U) {
            console->discard_line = true;
        } else {
            console->line[console->length++] = character;
        }
    }
}
```

9.6. Интерфейс доработанного автомата: traffic_light.h

```
#ifndef TRAFFIC_LIGHT_H
#define TRAFFIC_LIGHT_H

#include <stdbool.h>
#include <stdint.h>

enum {
    TRAFFIC_DEFAULT_RED_MS = 4000U,
    TRAFFIC_GREEN_MS = 1000U,
    TRAFFIC_BLINK_HALF_PERIOD_MS = 250U,
    TRAFFIC_BLINK_EDGE_COUNT = 8U,
    TRAFFIC_YELLOW_MS = 1000U,
    /* Keep every deadline interval below INT32_MAX milliseconds. */
    TRAFFIC_MAX_TIMEOUT_SECONDS = 2147483U
};

typedef enum {
    TRAFFIC_STATE_RED = 0,
    TRAFFIC_STATE_GREEN,
    TRAFFIC_STATE_BLINKING_GREEN,
    TRAFFIC_STATE_YELLOW
} TrafficLightState;

typedef void (*TrafficLightOutput)(bool red, bool yellow, bool green);

typedef struct {
    TrafficLightState state;
    uint32_t state_started_ms;
    uint32_t deadline_ms;
    uint32_t red_ms;
    uint8_t blink_edge;
    bool shorten_next_red;
    bool red_shortened;
    bool button_enabled;
    TrafficLightOutput output;
} TrafficLight;

void traffic_light_init(TrafficLight *light, uint32_t now_ms, TrafficLightOutput output);
void traffic_light_on_button_pressed(TrafficLight *light, uint32_t now_ms);
void traffic_light_update(TrafficLight *light, uint32_t now_ms);
bool traffic_light_set_timeout(TrafficLight *light, uint32_t seconds, uint32_t now_ms);
const char *traffic_light_state_name(const TrafficLight *light);

#endif
```

9.7. Автомат светофора и изменение тайм-аута: traffic_light.c

```
#include "traffic_light.h"

static bool deadline_reached(uint32_t now_ms, uint32_t deadline_ms)
{
    return (int32_t)(now_ms - deadline_ms) >= 0;
}

static void apply_output(const TrafficLight *light)
{
    const bool red = light->state == TRAFFIC_STATE_RED;
    const bool yellow = light->state == TRAFFIC_STATE_YELLOW;
    const bool green = light->state == TRAFFIC_STATE_GREEN
        || (light->state == TRAFFIC_STATE_BLINKING_GREEN && light->blink_edge % 2U == 0U);
    light->output(red, yellow, green);
}

static void enter_state(TrafficLight *light, TrafficLightState state, uint32_t now_ms)
{
    light->state = state;
    light->state_started_ms = now_ms;
    light->blink_edge = 0U;
    switch (state) {
        case TRAFFIC_STATE_RED:
            light->red_shortened = light->shorten_next_red;
            light->shorten_next_red = false;
            light->deadline_ms = now_ms + (light->red_shortened ? light->red_ms / 4U : light->red_ms);
            break;
        case TRAFFIC_STATE_GREEN:
            light->deadline_ms = now_ms + TRAFFIC_GREEN_MS;
            break;
        case TRAFFIC_STATE_BLINKING_GREEN:
```

```
            light->deadline_ms = now_ms + TRAFFIC_BLINK_HALF_PERIOD_MS;
            break;
        case TRAFFIC_STATE_YELLOW:
            light->deadline_ms = now_ms + TRAFFIC_YELLOW_MS;
            break;
        default:
            break;
    }
    apply_output(light);
}

void traffic_light_init(TrafficLight *light, uint32_t now_ms, TrafficLightOutput output)
{
    light->red_ms = TRAFFIC_DEFAULT_RED_MS;
    light->shorten_next_red = false;
    light->red_shortened = false;
    light->button_enabled = true;
    light->output = output;
    enter_state(light, TRAFFIC_STATE_RED, now_ms);
}

void traffic_light_on_button_pressed(TrafficLight *light, uint32_t now_ms)
{
    if (!light->button_enabled || light->state == TRAFFIC_STATE_GREEN) {
        return;
    }
    if (light->state == TRAFFIC_STATE_RED) {
        const uint32_t deadline = light->state_started_ms + light->red_ms / 4U;
        light->red_shortened = true;
        if (deadline_reached(now_ms, deadline)) {
            enter_state(light, TRAFFIC_STATE_GREEN, now_ms);
        } else {
            light->deadline_ms = deadline;
        }
    } else {
        light->shorten_next_red = true;
    }
}

void traffic_light_update(TrafficLight *light, uint32_t now_ms)
{
    for (uint32_t step = 0U; step < 16U; ++step) {
        if (!deadline_reached(now_ms, light->deadline_ms)) {
            return;
        }
        const uint32_t scheduled_ms = light->deadline_ms;
        switch (light->state) {
            case TRAFFIC_STATE_RED:
                enter_state(light, TRAFFIC_STATE_GREEN, scheduled_ms);
                break;
            case TRAFFIC_STATE_GREEN:
                enter_state(light, TRAFFIC_STATE_BLINKING_GREEN, scheduled_ms);
                break;
            case TRAFFIC_STATE_BLINKING_GREEN:
                ++light->blink_edge;
                if (light->blink_edge >= TRAFFIC_BLINK_EDGE_COUNT) {
                    enter_state(light, TRAFFIC_STATE_YELLOW, scheduled_ms);
                } else {
                    light->deadline_ms = scheduled_ms + TRAFFIC_BLINK_HALF_PERIOD_MS;
                    apply_output(light);
                }
                break;
            case TRAFFIC_STATE_YELLOW:
                enter_state(light, TRAFFIC_STATE_RED, scheduled_ms);
                break;
            default:
                enter_state(light, TRAFFIC_STATE_RED, now_ms);
                return;
        }
    }
}

bool traffic_light_set_timeout(TrafficLight *light, uint32_t seconds, uint32_t now_ms)
{
    if (seconds == 0U || seconds > TRAFFIC_MAX_TIMEOUT_SECONDS) {
        return false;
    }
    light->red_ms = seconds * 1000U;
    if (light->state == TRAFFIC_STATE_RED) {
        light->deadline_ms = light->state_started_ms
            + (light->red_shortened ? light->red_ms / 4U : light->red_ms);
        traffic_light_update(light, now_ms);
    }
    return true;
}

const char *traffic_light_state_name(const TrafficLight *light)
{
    switch (light->state) {
        case TRAFFIC_STATE_RED: return "red";
        case TRAFFIC_STATE_GREEN: return "green";
        case TRAFFIC_STATE_BLINKING_GREEN: return "blinking green";
        case TRAFFIC_STATE_YELLOW: return "yellow";
        default: return "unknown";
    }
}
```